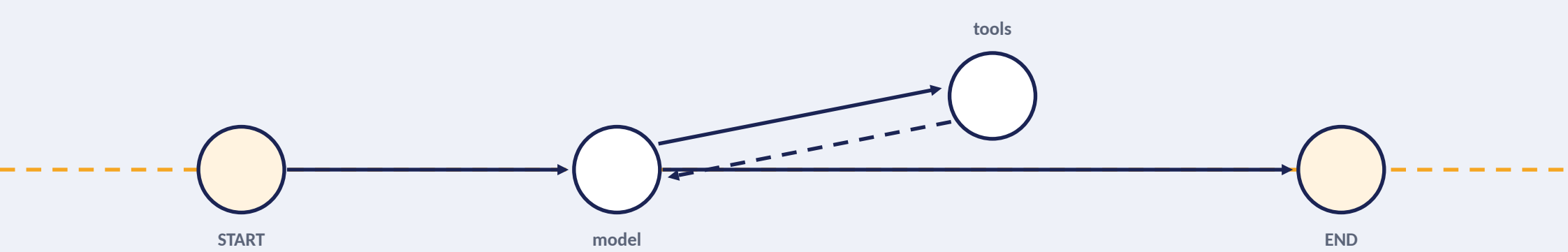
LANGGRAPH · CHAPTER 12 · FLASHCARDS

# Two Complete Agents

## *Agentic RAG and an email assistant*

12 code cards on a knowledge-base chatbot and an email agent that remembers



# Retrieval becomes a tool

*Classic RAG always searches once; agentic RAG lets the model decide if, what and how often*

- The model writes its own query, can search zero, one or several times, and can pick another tool.
- That freedom makes its choices worth checking: Chapter 14 grades the tools it called, not just the answer.

TEXT

```
Classic RAG: a fixed pipeline
  question -> retrieve top-k -> prompt + chunks -> answer
  one search, every time, on the question exactly as asked

Agentic RAG: the Chapter 7 agent loop, with search as one tool
  question -> model decides
     company data?  -> search_knowledge_base(a query it wrote) -> answer
     the weather?   -> get_weather(city), no search             -> answer
     small talk?    -> answers directly, no tool

Two phases in 9-Agentic-RAG/src/
  ingestion, once:   datasets/ -> loaders.py -> chunking.py -> vectorstore.py
  query, each turn:  app.py -> agents.py (create_agent) -> tools.py -> answer
```

INGESTION · RUNS ONCE

# Files to tagged chunks

*Rows become "column: value" text, text becomes overlapping chunks, each tagged with its file*

- Keeping column names beside values lets "who covers Europe?" match the row that says region: Europe.
- The 200-character overlap means a fact cut at a chunk boundary still survives whole in one chunk.

PYTHON

```python
# loaders.py: one line per CSV row, written as "column: value" pairs
def _load_csv(path: Path) -> str:
    with path.open("r", encoding="utf-8", newline="") as f:
        return "\n".join(", ".join(f"{key}: {value}" for key, value in row.items())
                         for row in csv.DictReader(f))

# chunking.py: 1,000-char windows, each starting 200 before the last ended
def fixed_size_chunk(text, chunk_size, chunk_overlap):
    step = chunk_size - chunk_overlap
    chunks = (text[start:start + chunk_size].strip()
              for start in range(0, len(text), step))
    return [chunk for chunk in chunks if chunk]

# pipeline.py: the source file travels with every chunk into Chroma
for doc in documents:
    for i, chunk in enumerate(fixed_size_chunk(doc.text, CHUNK_SIZE, CHUNK_OVERLAP)):
        ids.append(f"{doc.source}::{i}")
        texts.append(chunk)
        metadatas.append({"source": doc.source, "chunk_index": i})
collection.add(ids=ids, documents=texts, metadatas=metadatas)  # batches of 100
# ingested 16 chunks from 11 files
```

# A docstring that steers the search

*The tool returns the closest chunks, each tagged with its file; the docstring steers the model*

- The docstring is the model's only guide; the "say what an ID is" rule came from a Chapter 14 eval run.
- n_results is a parameter, so it is in the tool schema too: the model may ask for more or fewer than 4.

PYTHON

```python
@tool
def search_knowledge_base(query: str, n_results: int = 4) -> str:
    """Search the internal knowledge base (CRM exports, projects, revenue, proposal
    documents). Use it for any question about company data: accounts, opportunities,
    sales reps, projects, revenue or proposals. Write the query in words: to look up
    an ID, say what it is, such as "sales rep REP01", never the bare ID."""
    collection = get_collection()
    if collection.count() == 0:
        return "The knowledge base is empty."     # returned, not raised
    results = collection.query(query_texts=[query], n_results=n_results)
    documents, metadatas = results["documents"][0], results["metadatas"][0]
    if not documents:
        return "No relevant information found in the knowledge base."
    # [source: crm_exports/sales_reps.csv] lets the answer cite its file
    return "\n\n---\n\n".join(f"{SOURCE_TAG}{meta['source']}]\n{text}"
                              for text, meta in zip(documents, metadatas))


ALL_TOOLS = [search_knowledge_base, get_weather]   # get_weather: Open-Meteo, no key
```

# The agent in five lines

*create_agent builds the model-and-tools loop; the system prompt says when each tool applies*

- @lru_cache builds the graph once per process; every question after that reuses it.
- temperature=0 keeps the tool choice as repeatable as the model allows, which helps Chapter 14's grading.

PYTHON

```
SYSTEM_PROMPT = """You are a helpful assistant for a company's internal chatbot.
You have access to tools:
- search_knowledge_base: use it to answer questions about company data
  (CRM accounts, opportunities, sales reps, projects, revenue, proposals).
- get_weather: use it for weather questions.

Only call a tool when it is relevant to the question. Answer general questions
directly. Ground every answer about company data in the knowledge base results,
and say so when you could not find the information. ..."""


@lru_cache(maxsize=1)                       # one graph per process
def get_agent():
    model = ChatOpenAI(model=config.AGENT_MODEL, temperature=0)   # gpt-4o-mini
    return create_agent(model, tools=ALL_TOOLS, system_prompt=SYSTEM_PROMPT)


print(list(get_agent().get_graph().nodes))
# ['__start__', 'model', 'tools', '__end__']   <- Chapter 7's loop
```

# One function for every caller

*ask_with_tools returns the answer and the tools this turn called, in order*

- The chat UI and Chapter 14's evaluation both call it, so the evaluation tests the path users hit.
- The slice works because the system prompt is not stored in state: your messages come back first, unchanged.

PYTHON

```python
def ask_with_tools(question, chat_history=None, session_id=None):
    messages = [*(chat_history or []), {"role": "user", "content": question}]
    result = get_agent().invoke({"messages": messages}, config={
        "run_name": "chatbot",            # the trace's name in LangSmith
        "tags": ["chatbot"],
        # one LangSmith thread per chat (Chapter 13)
        "metadata": {"session_id": session_id} if session_id else {},
    })
    new_messages = result["messages"][len(messages):]   # this turn only
    tools = [m.name or "" for m in new_messages if isinstance(m, ToolMessage)]
    return result["messages"][-1].text, tools

# app.py: one session_id per browser chat
answer, tools = ask_with_tools(question, chat_history=st.session_state.messages,
                               session_id=st.session_state.session_id)

# Which sales rep covers Europe?           tools: ['search_knowledge_base']
# Which projects are delayed, by how much? tools: ['search_knowledge_base']
# What's the weather in London right now?  tools: ['get_weather']
```

# Three quiet RAG mistakes

*A duplicate tail chunk, a stale index and a bare-ID query all fail without an error*

- range(0, len(text), step) starts one window too many when the text ends inside the last overlap.
- Ingestion skips a non-empty collection, so edits to datasets/ never reach the agent until .chroma/ goes.

PYTHON

```python
# 1. The chunker can emit a tail that is already inside the previous chunk
fixed_size_chunk("x" * 1000, 1000, 200)     # lengths [1000, 200]: 200 repeated
# datasets/revenue/revenue.csv: 836 chars -> 2 chunks, the 2nd is 36 repeated chars
# a fix (not in the repo): never start a window inside the final overlap
#   range(0, max(len(text) - chunk_overlap, 1), step)    -> [836]

# 2. The index is built once; editing datasets/ changes nothing
#   $ python -m src.pipeline
#   ingestion skipped: the knowledge base already has chunks
#   -> delete .chroma/ and run it again

# 3. Embeddings match meaning, not codes: name what the ID is
search_knowledge_base.invoke({"query": "REP01"})            # missed the file
search_knowledge_base.invoke({"query": "sales rep REP01"})  # what the docstring asks

# Checked on langgraph 1.2.6 against the real datasets/ (no API calls)
```

# Classify, then route with Command

*A structured-output router labels each email; Command carries the update and the next node*

- Command[Literal[...]] is what get_graph() draws; without it the routing still runs but the picture is wrong.
- Only "respond" reaches the agent: notify ends like ignore here, where a real app would add a notify node.

PYTHON

```python
class Router(BaseModel):
    reasoning: str = Field(
        description="A concise explanation for the classification.")
    classification: Literal["ignore", "respond", "notify"]

llm_router = init_chat_model("openai:gpt-4o-mini").with_structured_output(Router)


def triage_router(state: State) -> Command[Literal["response_agent", "__end__"]]:
    email_input = state["email_input"]
    result = llm_router.invoke([
        {"role": "system", "content": system_prompt},   # John's profile + rules
        {"role": "user", "content": user_prompt},       # the email itself
    ])
    if result.classification == "respond":
        # one return value: the state update AND where to go next
        return Command(goto="response_agent", update={"messages": [
            {"role": "user", "content": f"Respond to the incoming email below.\n\n"
                                        f"{email_input}"}]})
    return Command(goto=END)            # ignore and notify both stop here

# 03_overall_email_agent.py
```

# An agent nested in a graph

*A compiled create_agent is one node; thread_id scopes the conversation, user_id the memories*

- Only START needs a static edge: triage_router's Command supplies every other transition.
- The checkpointer keys on thread_id; the Store's namespaces key on langgraph_user_id, across all threads.

PYTHON

```python
class State(TypedDict):
    email_input: dict                          # the email being handled
    messages: Annotated[list, add_messages]    # appended, never overwritten


builder = StateGraph(State)
builder.add_node("triage_router", triage_router)
builder.add_node("response_agent", build_response_agent())  # create_agent graph
builder.add_edge(START, "triage_router")       # Command does the rest
email_agent = builder.compile(checkpointer=MemorySaver(), store=store)


email_agent.invoke(
    {"email_input": initial_email, "messages": []},
    config={"configurable": {
        "thread_id": "alice-initial",   # checkpointer: this conversation's state
        "langgraph_user_id": "john",    # Store: John's memories, in every thread
    }},
)

# 05_full_agent_semantic_memory.py
```

MEMORY · SEMANTIC

# Facts the agent saves itself

*LangMem's two memory tools let the response agent store and search facts per user*

- "{langgraph_user_id}" is a template: LangMem fills it from config["configurable"] when a tool runs.
- InMemoryStore forgets everything when the process ends; a PostgresStore keeps the facts across restarts.

PYTHON

```python
store = InMemoryStore(index={"embed": "openai:text-embedding-3-small",
                             "dims": 1536})     # embeddings make search semantic

memory_namespace = ("email_assistant", "{langgraph_user_id}", "collection")
manage_memory_tool = create_manage_memory_tool(namespace=memory_namespace)
search_memory_tool = create_search_memory_tool(namespace=memory_namespace)


def build_response_agent():
    return create_agent(
        model="openai:gpt-4o-mini",
        tools=[write_email, schedule_meeting, check_calendar_availability,
               manage_memory_tool, search_memory_tool],  # the model decides
        system_prompt=system_prompt,
        store=store,
    )

# thread "alice-initial":   manage_memory saves Alice's /auth docs question
# thread "alice-follow-up": "Any update?" -> search_memory finds it,
#                           because both threads share langgraph_user_id "john"
```

MEMORY · EPISODIC

# Corrections as few-shot examples

*A human-corrected label is stored as an episode; triage retrieves similar ones by embedding*

- The application writes the episode from a person's label; the model never promotes its own guess.
- Search runs on meaning, so a differently worded vendor pitch can pull up last week's correction.

PYTHON

```python
def save_episodic_correction(user_id, email, label):
    """Write a trusted human correction as one episodic example."""
    store.put(("email_assistant", user_id, "examples"), str(uuid.uuid4()),
              {"email": email, "label": label, "source": "human_correction"})


# inside triage_router: find past corrections that look like this email
episodes = store.search(episodic_namespace(user_id),
                        query=str({"email": email}), limit=3)
system_prompt = triage_system_prompt.format(
    ...,
    examples=format_episodes(episodes),   # "Correct classification: IGNORE"
)


save_episodic_correction("john", corrected_vendor_email, "ignore")
run_email(agent, "john", "vendor-pitch", similar_vendor_email, "EPISODIC ...")
# Classification: ... (retrieved episodes: 1)

# 08_integrated_memory_agent.py
```

MEMORY · PROCEDURAL

# Rules that learn from feedback

*Each prompt section is a Store item by exact key; an optimizer rewrites it from feedback*

- index=False skips embedding: procedures are fetched by key, never searched, though they share the Store.
- The optimizer only proposes; the loop saves just the sections whose text changed, and leaves the rest.

PYTHON

```python
store.put(("email_assistant", user_id, "procedures"), "triage_notify",
          {"prompt": DEFAULT_PROCEDURES["triage_notify"], "source": "default"},
          index=False)                          # exact-key only, no embedding

procedure_optimizer = create_multi_prompt_optimizer("openai:gpt-4o-mini",
                                                    kind="prompt_memory")
feedback = "Ignore routine deployment FYIs when they explicitly say no action ..."
optimized = procedure_optimizer.invoke({
    "trajectories": [(feedback_trajectory, feedback)],
    "prompts": prompts,     # name, prompt, update_instructions, when_to_update
})

for old_prompt, new_prompt in zip(prompts, optimized):
    if new_prompt["prompt"] == old_prompt["prompt"]:
        continue                                # unchanged: keep it as it is
    store.put(namespace, PROMPT_SPECS[old_prompt["name"]][0],
              {"prompt": new_prompt["prompt"],
               "source": "optimized_from_user_feedback"}, index=False)

# the next email: triage_router re-reads every rule, so the change applies at once
```

# Three email-agent traps

*A missing user id fails the run, a reused thread piles up history, and nothing gates a send*

- Passing "messages": [] never clears a thread: the checkpointer restores it and add_messages appends.
- The tools only simulate; before a real write_email, pause for approval with interrupt() (Chapter 8).

PYTHON

```python
# 1. Every config needs langgraph_user_id, not just thread_id
email_agent.invoke(inputs, {"configurable": {"thread_id": "t1"}})
# 08: KeyError: 'langgraph_user_id'   (triage_router reads it first)
# 05: langmem.errors.ConfigurationError: Missing key in 'configurable'
#     field: langgraph_user_id. Available keys: ['thread_id']

# 2. One thread_id per email: a reused thread keeps the old messages
email_agent.invoke({"email_input": email_a, "messages": []}, cfg_t2)
email_agent.invoke({"email_input": email_b, "messages": []}, cfg_t2)
# messages: [email_a request, reply, email_b request, reply]

# 3. "SIMULATION ONLY": nothing is sent, and nothing asks first
@tool
def write_email(to: str, subject: str, content: str) -> str:
    """Simulate drafting an email without sending it."""
    return "SIMULATION ONLY: no email was sent. ..."

# 1 and 2 checked on langgraph 1.2.6 / langmem 0.0.30 with no model calls
```